Université Abdelmalek Essaâdi
École nationale des sciences appliquées
d'Al Hoceima

Fliére: Transformation Digitale et IA

Compte Rendu :
TP5

Réalisé par:
Amal Dadda Amami

Encadré par:
Prof.M.Cherradi

Année Universitaire:
2025/2026

1. Objectif

L'objectif de ce TP est de developper une application web en Jakarta EE avec l'architecture MVC2, Hibernate ORM pour la persistance MySQL, et un systeme d'authentification avec roles. L'application permet :

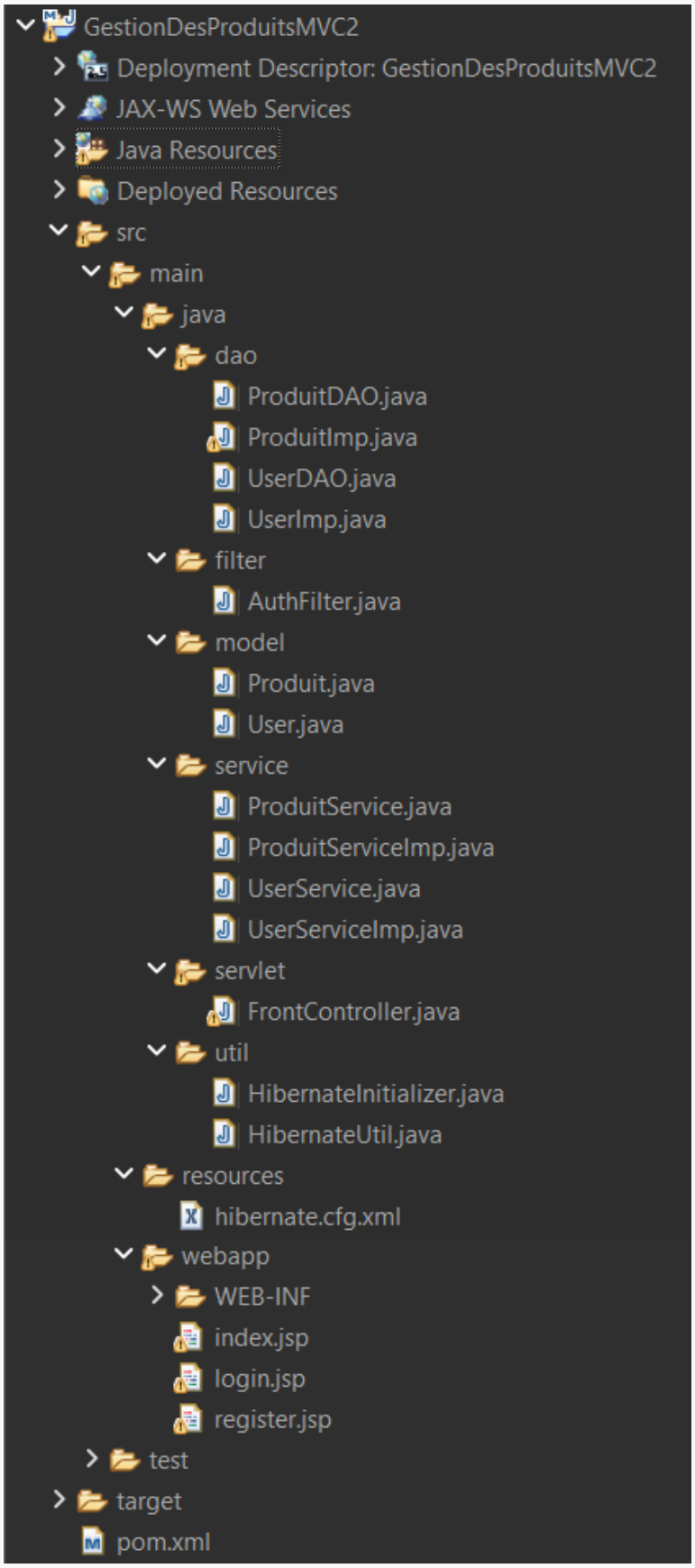
- Inscription et connexion avec gestion de session
- Affichage de la liste des produits pour tous les utilisateurs connectes
- Ajout, modification, suppression de produits (admin uniquement)
- Protection via AuthFilter avec verification session et roles
- Persistance reelle dans MySQL via Hibernate

2. Architecture du projet

La difference fondamentale avec MVC1 est l'utilisation d'un seul FrontController qui centralise toutes les requetes et les dispatche selon le parametre action.

Package	Fichiers	Role
model	Produit.java, User.java	Entites JPA mappees sur tables MySQL
dao	ProduitDAOImpl, UserImp	Acces BDD via Hibernate Session
service	ProduitServiceImp, UserServiceImp	Logique metier Singleton
servlet	FrontController.java	UN seul controleur pour toutes actions
filter	AuthFilter.java	Securite : session + verification roles
util	HibernateUtil, HibernateInitializer	SessionFactory + init demarrage
webapp (view)	login.jsp, register.jsp, index.jsp	Interface Bootstrap + Scriptlets

3. Structure du projet Maven:



4. Dépendances du projet

Dépendance	Version	Role
jakarta.servlet-api	6.0.0	API Servlet (scope provided)
jakarta.servlet.jsp.jstl GlassFish	2.0.0	JSTL pour Tomcat 10+
hibernate-core	6.4.0.Final	ORM Hibernate
mysql-connector-j	8.3.0	Driver JDBC MySQL

5. Configuration Hibernate

5.1 hibernate.cfg.xml

Fichier dans src/main/resources/ qui configure la connexion MySQL et la strategie de creation des tables. La propriete hbm2ddl.auto=update cree automatiquement les tables au demarrage si elles n'existent pas.

5.2 HibernateUtil.java

Initialise la SessionFactory une seule fois dans un bloc static. La SessionFactory est couteuse a creer et doit etre unique dans toute l'application.

5.2 HibernateInitializer.java

Force l'initialisation de Hibernate au demarrage de Tomcat. Sans ce listener, Hibernate ne s'initialise qu'au premier acces HTTP, ce qui causait des problemes avec l'AuthFilter qui s'executait avant le FrontController.

6. Entites JPA

Les entites utilisent les annotations Jakarta Persistence pour le mapping objet-relationnel.

- **@Entity :** mappe la classe sur une table
- **@Id:** designe la cle primaire
- **@GeneratedValue(strategy = GenerationType.IDENTITY):** delègue la generation a MySQL (AUTO_INCREMENT).

6-1 model/Produit.java :

```java
import jakarta.persistence.Column;

@Entity
@Table(name = "produits")
public class Produit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id ;

    @Column(nullable = false)
    private String nom ;

    @Column
    private String description;

    @Column(nullable = false)
    private double prix;
```

6-2 model/User.java :

```java
package model;

import jakarta.persistence.Column;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue( strategy = GenerationType.IDENTITY )
    private int id;

    @Column(nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String role;
```

7. Couche DAO avec Hibernate

Chaque methode DAO utilise try-with-resources pour garantir la fermeture automatique de la Session. Les operations d'ecriture utilisent une Transaction avec rollback en cas d'erreur.

8. FrontController

Unique Servlet mappee sur /app. Le parametre action determine l'operation via un switch. Le parametre est transmis via un champ cache dans les formulaires JSP.

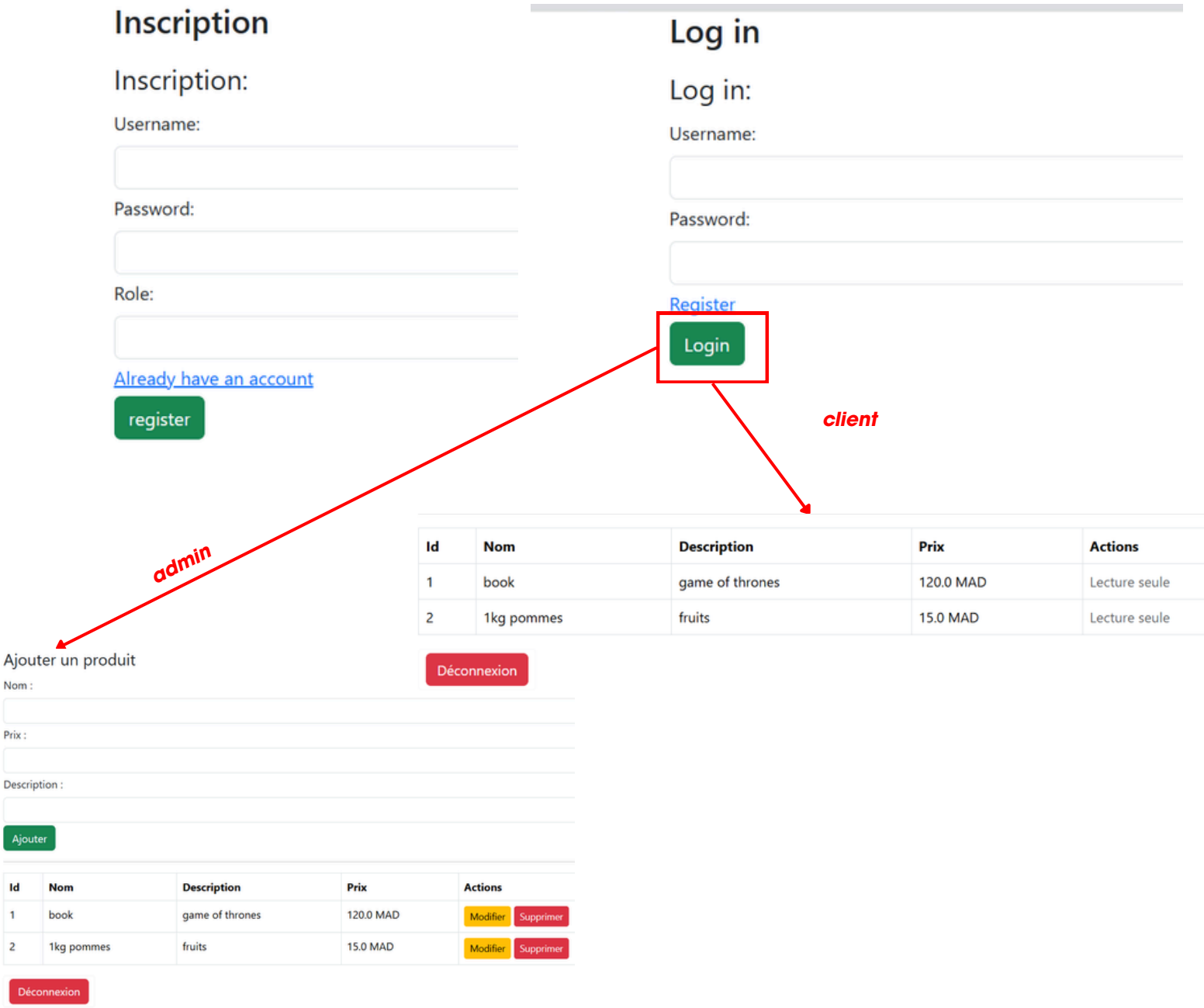
Action	HTTP	Operation	Vers
list	GET	getAllProduits()	Forward index.jsp
add	POST	addProduit()	Redirect app?action=list
edit	GET	getById() -> formulaire pre-rempli	Forward index.jsp
update	POST	updateProduit()	Redirect app?action=list
delete	GET	deleteProduit()	Redirect app?action=list
login	POST	loginUser()	Redirect app?action=list
register	POST	registerUser()	Redirect login.jsp
logout	GET	session.invalidate()	Redirect login.jsp

9. AuthFilter

Le filtre intercepte /app et applique deux niveaux de securite : verification de la session puis verification du role pour les actions admin.

Action	Non connecte	Client	Admin
login, register	Autorise	Autorise	Autorise
list	Redirect login.jsp	Autorise	Autorise
add, edit, update, delete	Redirect login.jsp	Redirect list	Autorise

10. Flux de l'application



11. Conclusion

Ce TP m'a permis de developper une application web complete en Jakarta EE avec l'architecture MVC2 et Hibernate ORM. Les concepts cles mis en pratique sont : le patron FrontController avec dispatch par parametre action, la persistance avec Hibernate (annotations JPA, Session, Transaction, HQL), l'initialisation au demarrage via @WebListener et la securisation centralisee avec AuthFilter verifiant session et roles.